Kerala Climate Network — Vyuham (Complex System & Network)



Extreme-rainfall climate network over Kerala (+ Western Ghats buffer) with two studies and a web dashboard:

- **Study A — Basin robustness:** which cells and basins matter most for network connectivity when removed.
- **Study B — Landslide validation:** does network centrality relate to 2018 landslide density, after accounting for slope.

Design principle: **the Python pipeline writes static JSON/GeoJSON files; the web app only reads them.** No backend server. The data files are the contract between the two halves, so the pipeline and the UI can be built independently (for example, one in Codex and one in Antigravity).

---

1. Architecture

```
IMD rainfall ─┐
Basins (WRIS) ─┤
SRTM DEM ──────┼─► pipeline (Python) ─► web/public/data/*.json ─► React dashboard (static)
Landslide inv.┘
```

Everything is reproducible from `config.yaml` and one command: `make all`.

2. Repository layout

```
kerala-climate-network/
├── AGENTS.md                # rules for coding agents (see section 9)
├── README.md
├── Makefile                 # make data | events | es | network | robustness | landslide | export | all
├── config.yaml              # all parameters (bbox, years, thresholds, seeds)
├── requirements.txt
├── data/
│   ├── raw/                 # downloaded, never edited
```

```
│   ├── interim/              # .npy / .parquet intermediates
│   └── processed/            # final tables
├── src/ktnet/
│   ├── io.py                 # loaders (IMD, shapefiles, DEM, inventory)
│   ├── grid.py               # cell table, cell↔basin assignment, cell area
│   ├── events.py             # extreme-event extraction
│   ├── sync.py               # Event Synchronization (numba)
│   ├── network.py            # thresholding, graph, centralities
│   ├── robustness.py         # node and basin removal experiments
│   ├── landslide.py          # inventory→cell density, slope, statistics
│   └── export.py             # writes web/public/data/*
├── pipeline/                 # thin scripts, one per stage, calling src/ktnet
├── tests/                    # pytest, see section 8
├── notebooks/                # exploration only, never the source of truth
└── web/                      # React + Vite + TypeScript dashboard
    ├── public/data/          # generated files land here
    └── src/
        ├── pages/            # Overview, Network, Robustness, Basins, Landslide, Method
        ├── components/       # MapView, MetricSelect, LineChartCard, StatCard, DataTable
        ├── state/            # shared filter state (metric, threshold, selected cell)
        ├── lib/              # data loaders, colour scales, types
        └── styles/
```

3. Pipeline stages (scientific spec)

Defaults below are **starting points; keep every one in `config.yaml`**.

Stage	What it does	Output
1 Data	Download IMD 0.25° daily rainfall (`imdlib`), 1991–2020. Load basin polygons (India-WRIS), SRTM 30 m DEM, 2018 landslide inventory (van Westen 2020, DOI 10.17026/dans-x6c-y7x2)	`data/raw/*`
2 Grid	Subset to bbox (~8°–13.5°N, 74°–78°E). Drop ocean and no-data cells. Build a cell table: `cell_id, lat, lon, area_km2`. Assign each cell to a basin by centroid-in-polygon, else `"other"`	`cells.parquet`
3 Events	JJAS only. Wet day = rain ≥ 1 mm. Per-cell threshold = 95th percentile of that cell's wet-day rainfall. Event = day above threshold. Use a continuous day index over all years so gaps between seasons are large	`events.npz`
4 Sync	Event Synchronization (Quian Quiroga et al. 2002) with dynamic lag, symmetric normalised score `Q_ij`. Implement in **numba**. Cap the maximum lag (config, start at ~10 days). Save the full N×N matrix	`es_matrix.npy`
5 Network	Keep the top 5% of `Q_ij` as links (config). No self-links. Undirected, weighted. Compute degree, betweenness, clustering, eigenvector centrality (use a `max_iter` fallback if it fails to converge)	`network.graphml`, `metrics.parquet`
6 Robustness (Study A)	Remove nodes by: random (100 runs, mean ± std), degree, betweenness. Track largest-connected-component fraction and global efficiency for 0–50% removed. Then remove **whole basin groups** and rank basins by impact. Repeat for thresholds 90/95/97 percentile	`robustness.parquet`, `basin_impact.parquet`, `sensitivity.parquet`
7 Landslide (Study B)	Count 2018 inventory points per cell → density per km². Mean and max slope per cell from the DEM. Spearman correlation of each centrality vs density, overall and stratified by slope class (tertiles). Bootstrap confidence intervals. Also count extreme days per cell in Aug 2018	`landslide_stats.parquet`
8 Export	Convert everything to the data contract below	`web/public/data/*`

4. Data contract (pipeline → UI)

All files in `web/public/data/`. Keep each under ~5 MB.

```
meta.json            { study_area, period, season, event_percentile, link_fraction,
                       n_cells, n_links, generated_at, config_hash }

cells.geojson        FeatureCollection of cell polygons. properties:
                       cell_id, lat, lon, basin_id, mean_slope, max_slope,
                       landslide_count_2018, landslide_density,
                       extreme_days_aug2018,
                       degree, betweenness, clustering, eigenvector

links.json           [ { source, target, weight } ]        // thresholded network only

basins.geojson       basin polygons. properties: basin_id, name, n_cells, impact_rank,
                       lcc_drop_if_removed, efficiency_drop_if_removed

robustness.json      { strategies: { random:{mean:[],std:[]}, degree:[], betweenness:[] },
                       fractions: [0,0.01,...,0.5],
                       metrics: ["lcc","efficiency"] }
                     // one series per metric per strategy

critical_nodes.json  top 15 cells by removal impact: cell_id, rank, lcc_drop, basin_id

sensitivity.json     per threshold (90/95/97): overlap of top-15 critical nodes,
                     rank correlation of centralities

landslide_stats.json { overall: {metric: {rho, p, ci_low, ci_high}},
                       by_slope_class: [ {class, n, metric, rho, p} ],
                       scatter: [ {cell_id, centrality, density, slope} ] }

landslide_points.json  optional: 2018 inventory points (lat, lon), thinned if large
```

Rule: **if the UI needs a new field, change the contract first, then both sides.**

5. Dashboard design

Clean, calm, data-first. Light theme by default with a dark toggle.

Style

- One accent colour, neutral greys elsewhere, generous spacing, rounded cards.
- Sequential colour scale (viridis-like) for centrality, diverging only where needed.
- System font or Inter. Charts with light gridlines and direct labels.
- Fully responsive, map on top on mobile.

Layout

- Left sidebar navigation (collapses to a tab bar on mobile).
- Top bar with global controls: metric selector, threshold (90 / 95 / 97), link-weight slider.
- Main area: map card + chart/table cards. Right-side detail panel that opens when a cell or basin is clicked.

Pages

1. **Overview** — 4 stat cards (cells, links, top basin by impact, strongest correlation), a map coloured by the chosen metric, and two short "key findings" sentences generated from the data.

2. **Network Explorer** — map of cells coloured by metric; toggle links on/off; slider to show only links above a weight; click a cell to see its metrics, basin, slope, extreme days.
3. **Robustness (Study A)** — line chart of LCC and efficiency vs fraction removed for random / degree / betweenness (tabs for the two metrics); slider that greys out removed nodes on the map; table of the top 15 critical nodes with a highlight-on-map action.
4. **Basins** — bar chart of basin impact ranking; map with basin polygons coloured by impact; click a basin to see its cells and their centralities.
5. **Landslide validation (Study B)** — map with centrality layer and optional 2018 landslide points; scatter of centrality vs landslide density coloured by slope class; table of correlations (overall and by slope class); toggle for Aug 2018 extreme-day layer.
6. **Method & limits** — plain description of the pipeline, all parameters from `meta.json`, and the limitation list below.

Tech

- React + Vite + TypeScript, Tailwind CSS
- MapLibre GL JS (or Leaflet) for maps, Recharts (or Plotly) for charts
- No backend; `fetch('/data/…')` on load; a small typed loader in `lib/`
- Build with `npm run build`; the output is static and can be hosted anywhere (GitHub Pages, Netlify)

6. Build order (10 days)

Day	Work	Done when
1	Repo scaffold, `config.yaml`, data download, IMD subset and map	rainfall map plotted, all raw data present
2	Grid table, basin assignment, event extraction	`cells.parquet` and `events.npz` exist
3	Event Synchronization in numba, tested	`es_matrix.npy` passes tests
4	Network + metrics + export of `cells.geojson`, `links.json`; **start the web scaffold**	map shows centrality from real data
5	Node-removal experiments, `robustness.json`, Robustness page	curves display in the UI
6	Basin removal, sensitivity runs, Basins page	basin ranking displayed
7	Landslide density and slope per cell	`landslide_stats` input table complete
8	Correlations, Aug 2018 layer, Landslide page	scatter and table displayed
9	Overview and Method pages, polish, README, screenshots	full dashboard runs from static files
10	Buffer, bug fixes, final build	`make all && npm run build` works from scratch

Cut lines: if behind by Day 5, cut sensitivity to one extra threshold. If behind by Day 8, show Study B as a descriptive map only (no statistics). If both are at risk, finish Study A and treat Study B as future work.

7. Working with the two tools

Suggested split (adjust as you like):

- **Tool 1: pipeline** — everything under `src/`, `pipeline/`, `tests/`, `Makefile`.
- **Tool 2: dashboard** — everything under `web/`.
- The only shared surface is the data contract in section 4. Give both tools this file.
- For UI work before the pipeline is ready, have the UI tool generate **mock data files that follow the contract exactly** and swap in real ones later.

Give each tool **one stage at a time**, ask it to run and show results before moving on, and commit after each stage. Example first prompts:

> **Pipeline, prompt 1:** "Read AGENTS.md and sections 2–4 of the spec. Scaffold the repo layout, `config.yaml`, `requirements.txt` and `Makefile`. Implement stage 1 (data download and loading) only. Do not implement later stages. Show me what files were created and how to run it."
>
> **Pipeline, prompt 3:** "Implement `sync.py` with numba: Event Synchronization with dynamic lag and symmetric normalised Q. Add pytest tests: symmetry, Q between 0 and 1, identical series give Q≈1, independent random series give low Q. Run on a 20-cell subset and report the runtime."
>
> **UI, prompt 1:** "Read section 5 and the data contract in section 4. Scaffold a Vite + React + TypeScript + Tailwind app. Create typed loaders for every file in the contract and a mock-data generator that writes valid files into `web/public/data/`. Build the app shell (sidebar, top bar, theme toggle) and the Overview page only."

8. Tests and checks

- ES is symmetric; Q in [0, 1]; identical series → high Q; independent random series → low Q.
- Extreme-event counts per cell are roughly 5% of wet days (sanity check on the percentile).
- Graph has the expected node count; link count matches the chosen fraction.
- Removal curves start at 1.0 (LCC and efficiency normalised to the intact network) and decrease.
- Landslide points map to cells: total points in cells ≤ total inventory points (some fall outside the grid).
- Contract test: every exported file validates against its schema, and the UI loaders parse them.
- Fixed random seeds; the same config gives identical output.

9. AGENTS.md (paste into the repo root)

```
# Rules for coding agents

- Read the spec before writing code. Work on ONE stage or page at a time.
- Never invent data or results. If a dataset is missing, stop and say so.
- Every parameter goes in config.yaml. No magic numbers in code.
- Do not change the data contract without updating both the pipeline and the UI.
- Write tests for numerical code. Run them and report the results.
- Run new code on a small subset before the full dataset and report the runtime.
- Keep functions small and typed. No notebooks as source of truth.
- Log data provenance (source, download date, version) to data/raw/PROVENANCE.md.
- After each stage: list files changed, how to run it, and what was verified.
- When uncertain about a scientific choice, ask; do not silently pick one.
```

10. Limits to state in the dashboard's Method page

- IMD grid cells are interpolated from gauges, not physical stations. Results point to regions of interest for monitoring, not exact gauge locations.
- At 0.25° (~27 km), Kerala has only about 50 cells. The buffer is what makes the network large enough to analyse; basin-level results are approximate.
- Landslide density depends strongly on slope, lithology and land use. Study B is exploratory, and a moderate or weak correlation is a valid finding.
- Removal experiments show structural importance in the network, not physical flood or landslide risk.
- Results depend on the event percentile and link fraction; the sensitivity analysis quantifies this.

11. Checklist for the final report or slides

- Problem and question for each study (one sentence each)
- Data table with sources and dates
- Method figure (the pipeline diagram in section 1 works)
- 4–5 result figures: centrality map, degradation curves, basin ranking, landslide scatter, sensitivity summary
- Limits (section 10)
- Dashboard screenshots or a hosted link